

Segmento *DIGITAL*:
MOBILE

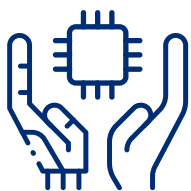
IMPACTO
por **BCP**



Capibaras en Acción
Entendiendo los Principios
SOLID

¿Qué son los principios SOLID?

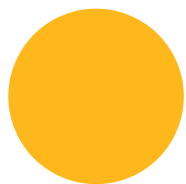
*Es un conjunto de **5** principios que ayudan a escribir código bien organizado, fácil de entender y mantener.*



Aplicarlos mejora la calidad de nuestros programas sin complicarlos.

¿Qué veremos hoy?

*Exploraremos los principios **SOLID** a través de ilustraciones con capibaras estilo Ghibli, para entender cada concepto de forma simple y visual.*





Responsabilidad Única (Single Responsibility Principle)




```
// ❌ Clase con múltiples responsabilidades
class CapibaraMultitask {
    fun regar() { /* ... */ }
    fun cocinar() { /* ... */ }
    fun limpiar() { /* ... */ }
}
```

```
// ✅ Separación de responsabilidades
class CapibaraJardinera {
    fun regar() { /* riega plantas */ }
}
```

```
class CapibaraCocinera {
    fun cocinar() { /* cocina con amor */ }
}
```

```
class CapibaraLimpiadora {
    fun limpiar() { /* deja todo reluciente */ }
}
```

En el código, cada clase debe encargarse de una sola cosa.





Abierto/Cerrado (Open/Closed Principle)

No cambié lo que era bueno,
solo agregué algo mejor.



```
// Clase base: Caña de pescar normal (ya funciona bien)
open class CanaDePescar {
    open fun pescar() {
        println("Capibara pesca con caña básica 🪝")
    }
}
```

```
// Nueva funcionalidad: caña con magia ✨
// (se extiende, no se modifica la original)
class CanaDePescarMagica : CanaDePescar() {
    override fun pescar() {
        super.pescar()
        println("¡Ahora lanza chispas mágicas y atrae más peces! 🐟✨")
    }
}
```

*Así debe ser el
código:
Se puede
extender sin
modificar lo que
ya funciona.*





Sustitución de Liskov (Liskov Substitution Principle)



```
// Clase base
open class Capibara {
    open fun deslizarse() {
        println("La capibara se desliza alegremente sobre el hielo 🧊")
    }
}

// Subclase válida que mantiene el comportamiento
class CapibaraDelBosque : Capibara()

// Subclase inválida que rompe el comportamiento esperado
class CapibaraConAlas : Capibara() {
    override fun deslizarse() {
        throw UnsupportedOperationException("¡Yo vuelo, no me deslizo!")
    }
}
```

*Las subclases
deben
comportarse
como sus clases
padre, sin romper
nada.*





Segregación de Interfaces (Interface Segregation Principle)



```
// ✗ Esta interfaz obliga a implementar métodos innecesarios
interface CapibaraTodoEnUno {
    fun pescar()        // Tiene una cuerda
    fun escalar()       // Tiene equipo de escalada
    fun volar()         // Tiene un paracaídas
    fun explorar()      // Lleva mapa, brújula, o actitud de aventura
}
```

```
// ✔ Diseño correcto con interfaces específicas
```

```
interface Pescadora {
    fun pescar()
}
```

```
interface Escaladora {
    fun escalar()
}
```

```
interface Paracaidista {
    fun volar()
}
```

```
interface Profesora {
    fun enseñar()
}
```

```
interface Exploradora {
    fun explorar()
}
```

Una interfaz debe implementar solo lo que realmente necesita.



Inversión de Dependencias (Dependency Inversion Principle)




```
// Abstracción: define cómo se bebe agua, sin importar de dónde venga
interface FuenteDeAgua {
    fun beber()
}

// Capibara que bebe de una botella
class BotellaAgua : FuenteDeAgua {
    override fun beber() {
        println("Capibara bebe agua de una botella 🍷")
    }
}

// Capibara que bebe del río
class Rio : FuenteDeAgua {
    override fun beber() {
        println("Capibara bebe agua del río 🌊")
    }
}

// Capibara que bebe de un cuenco
class Cuenco : FuenteDeAgua {
    override fun beber() {
        println("Capibara bebe agua del cuenco 🍲")
    }
}
```

Así como en el dibujo: “No importa de dónde venga el agua, mientras sea agua”, en el código: no importa de qué clase venga la funcionalidad, mientras cumpla con lo esperado.

**No se trata solo de programar
bien.**



**Se trata de construir como los
capibaras:
con calma, claridad y propósito.**

**Cada principio SOLID no es una
regla rígida... Es una forma de
cuidar tu código y a quienes
lo usarán después.**

**Haz que tu código sea un lugar
donde todos quieran estar.**

